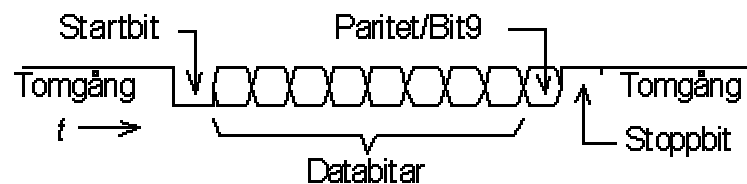
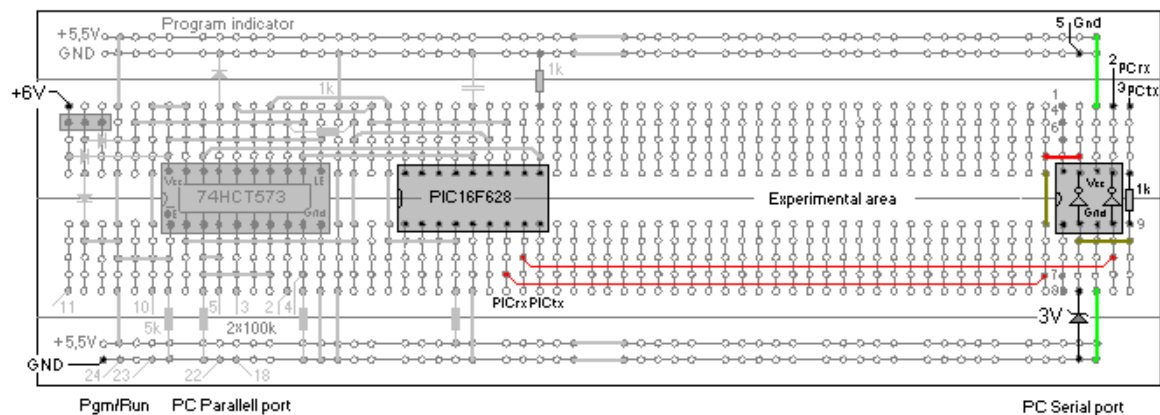


Maskinnära programmering 6B2266

Häfte: Seriekommunikation, hantering av textsträngar och talsträngar.

Programexempel med 16F84 och 16F628.



- Datakommunikation, seriell kommunikation
- `getchar()` och `putchar()`
- Testprogrammet `echo.c`
- Hello World! `hello.c`
- Jämföra strängar. `cmpcom.c`
- Använda flash-minnet. `rememb.c`
- Skriva ut variabler. `numtoa.c`
- Mata in tal. `atochar.c`

© 2005 William Sandqvist

Datakommunikation

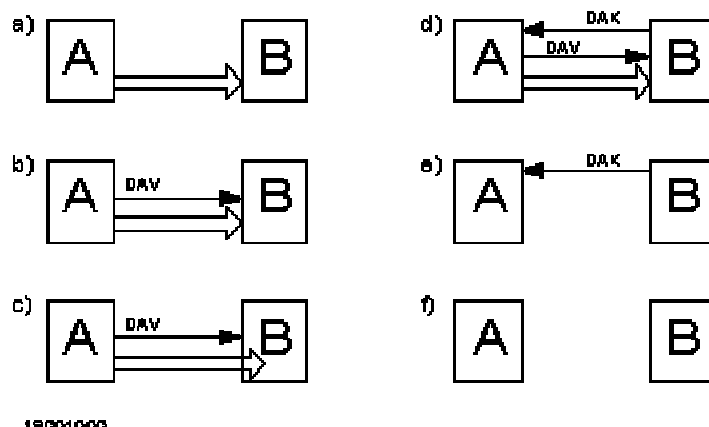
Parallell kommunikation

Överföring av data mellan en mikrostyrcikret och olika yttre enheter sker ofta via mikrostyrcikretsens parallella I/O-portar. Eftersom portarna är dubbelriktade är de lämpade att både sända och ta emot data "byte för byte". Denna dataöverföring kan karaktäriseras som "bit parallell, byte seriell".

Ett grundläggande problem vid mottagningen av data är att kunna ta reda på när en byte på en port övergår i den nästföljande. Vid växlingen mellan två bytes kommer nämligen inte de 8 bitarna att anta stabila värden exakt samtidigt. Den som mottager portens 8 bitar kan komma att misstolka övergångstillstånden som ett antal överförda bytes.

Ett praktiskt fungerande system för asynkron parallell kommunikation använder två styrsignaler och en teknik som kallas för **Handskakning**. De två styrsignalerna brycks kallas för **DAV** (**D**ata**A**vailable, data tillgängligt) och **DAK** (**D**ata **A**c**K**nowledge, data bekräftelse).

Vid så kallad *automatisk handskakning* finns dessa styrsignaler inbyggda i mikrostyrcikretsens port, i annat fall låter man två bitar ur någon annan port utföra denna funktion.



Handskakningsproceduren vid parallell dataöverföring från A till B.

Figuren illustrerar steg för steg hur dataöverföring av en byte från enhet A till B går till. Det hela börjar med att A presenterar en byte för B (steg a). När data är stabilt signalerar A detta till B med **DAV** = 1 (steg b). B upptäcker detta och mottager byten (steg c). Med **DAK** = 1 kvitterar B till A att data mottagits (steg d). A kan nu ta bort byten och sätta **DAV** = 0 (steg e). I sista steget (steg f) upptäcker B att **DAV** = 0 och svarar i sin tur med att sätta **DAK** = 0.

Sekvensen kan nu återupprepas från steg a.

Handskakningstekniken tillåter olika snabba enheter att kommunicera med varandra. Varje enhet väntar den tid som behövs för att den andra ska hinna svara, vilket möjliggör kommunikation med en godtyckligt långsam enhet. Å andra sidan tillåts även snabba enheter att kommunicera så snabbt de kan med varandra, vilket innebär att handskakningstekniken även kan möjliggöra mycket höga överföringshastigheter. (Detta gäller speciellt mellan mikrostyrcikretar som har automatisk handskakning inbyggt i portarna).

Seriell kommunikation

När man ska överföra data över längre sträckor är seriell dataöverföring ofta att föredraga framför parallell. Istället för att överföra en byte över 8 parallella ledningar (egentligen 11 ledningar om man räknar **GND**, **DAV** och **DAK**), kan man vid seriell kommunikation överföra samma byte med en bit i taget över 1 ledning (egentligen 2 ledningar om man inkluderar återledaren **GND**). Besparingen av antalet ledningar får man "betala" med sänkt överföringshastighet.

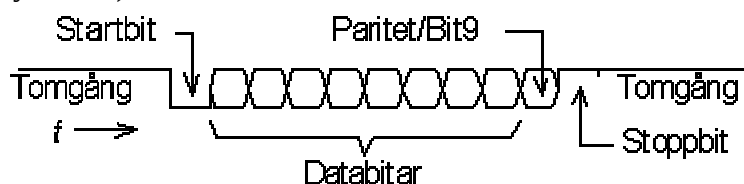
Ibland kan det vara tillräckligt att kunna överföra data i en riktning. Detta brukar kallas för **simplex**. I de flesta fall är emellertid en två-vägs kommunikation nödvändig, tex. för att sändaren av meddelandet åtminstone ska kunna få en bekräftelse på att mottagningen gått bra (eller, om den misslyckats, få en begäran om omsändning). Detta brukar kallas för **duplex**. Vid full duplex kan kommunikationen pågå samtidigt i båda riktningarna. Vid halv duplex kan man bara kommunicera i en riktning i taget.

Vid seriekommunikation är det nödvändigt att sändare och mottagare opererar med samma i förväg överenskomna hastighet. Den hastighet med vilken bitarna överförs kallas för **Baudrate** [bit/sek]. Vanliga Baudrate's är multiplar av 75 bit/sek som: 75, 150, 300, 600, 1200, 9600, 19200 och 38400 bit/sek.

Seriell kommunikation kan karakteriseras som "bit seriell, byte seriell". Ett grundläggande problem vid mottagandet av data är att avgöra var gränsen för en viss byte går i en ström av seriella bitar. Detta löses på helt olika sätt vid asynkron seriekommunikation och vid synkron seriekommunikation.

Asynkron seriekommunikation

Den asynkrona överföringstekniken innebär att man för varje byte lägger till extra bitar som ska göra det möjligt att skilja ut byten från bitströmmen. Ofta lägger man dessutom till en bit för felindikering. En nackdel med detta arrangemang är att dessa extra bitar sänker den effektiva överföringshastigheten (det vill säga antalet överförda bytes/sek).



Överföringsformatet vid seriell datakommunikation.

Utgångsläget inför överföringen av en byte är alltid att datalinjen är hög "1". Detta är "tomgång" (idle mode). Dataöverföringens start signaleras med att datalinjen hålls låg "0" under ett tidsintervall som är en bit långt ($T = 1/\text{Baudrate}$). Detta är *startbiten*.

Under 8 lika långa tidsintervall följer sedan databitarna, ettor eller nollor, med den minst signifikanta biten först och den mest signifikanta biten sist. Därefter följer en *paritetsbit*, ett hjälpmedel vid detekteringen av överföringsfel (detta begrepp kommer att förklaras senare). Överföringen avslutas slutligen av att datalinjen för åtminstone ett bit-tidsintervall är hög. Det är *stoppbiten*. Efter detta är datalinjen åter i tomgång. Överföringen av nästa byte kan nu följa omedelbart, eller dröja mycket länge.

Det är kännetecknande för asynkron datakommunikation att bytes kan överföras i valfri takt upp till den maximala överföringshastigheten.

Mottagningen av data sker i princip genom att man först inväntar startbitens negativa flank, för att därefter registrera datalinjen vid tidpunkterna $3T/2$, $5T/2$, ... $17T/2$ relativt startflanken (dessa tider är databitarnas "mittpunkter"). Mottagaren "synkroniseras" om på nytt av varje startflank. Detta medför att relativt stora avvikelser från överenskommen bauderate kan förekomma, hos både mottagare och sändare, innan dataöverföringen misslyckas.

Det förekommer många varianter av det ovan beskrivna asynkrona seriella överföringsformatet. Så kan tex. en byte innehålla 8, 7, 6, eller 5 databitar som överförs. Vidare kan den så kallade paritetsbiten ofta uteslutas, eller som vi senare ska se, bytas ut mot en 9:e bit med en speciell betydelse. Slutligen kan 1, 2,5 eller 2 stoppbitar användas. Det är således en hel mängd olika parametrar som sändare och mottagare måste ena sig om innan dataöverföringen kan äga rum!

Paritetsbiten

Vid dataöverföring över större avstånd måste man alltid räkna med möjligheten att dataöverföringen störs. En sedan länge använd metod för feldetektering är den s.k. paritetskontrollen. Till den byte som överförs lägger man en extra s.k. *paritetsbit* som är så vald att det totala antalet ettor är jämnt vid *jämn paritet* (J), eller udda vid *udda paritet* (U). Sedan det överenskommits om vilken paritet J/U som ska gälla, kan mottagaren upptäcka om någon av de överförda bitarna "bytt" värde till följd av en störning. (Om störningen drabbat två bitar kan dock felet passera obemärkt).

Start och stoppbitar ingår inte vid beräkningen av paritetsbiten.

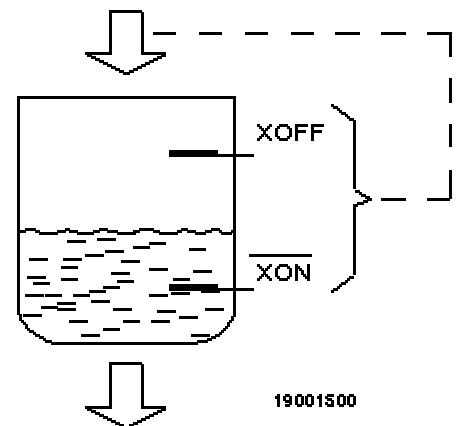
Paritetsbiten härstammar från automatiseringsteknikens barndom, då data överfördes med stansade hålremsor av pappersmaterial. De mekaniska stansutrustningarna "hängde sig" ibland, men sannolikheten för att två stanspinnar skulle kärva samtidigt var obefintlig, så paritetskontrollen utgjorde då ett fullgott skydd mot felstansade koder.

En modernare metod för att gardera sig mot dataöverföringsfel går ut på att beräkna en kontrollsumma ur det block av bytes som ska överföras (ett block är tex 256 bytes). Denna kontrollsumma översändes sedan tillsammans med datablocket. På mottagar jämför man kontrollsumman med den kontrollsumma man själv kan räkna fram ur de data man mottagit. Man kan med kontrollsummemetoder upptäcka överföringsfel med färre överförda kontrollbitar än vad som är fallet med paritetskontroll. Paritetskontroll är trots detta en vanlig metod för feldetektering.

Seriell handskakning

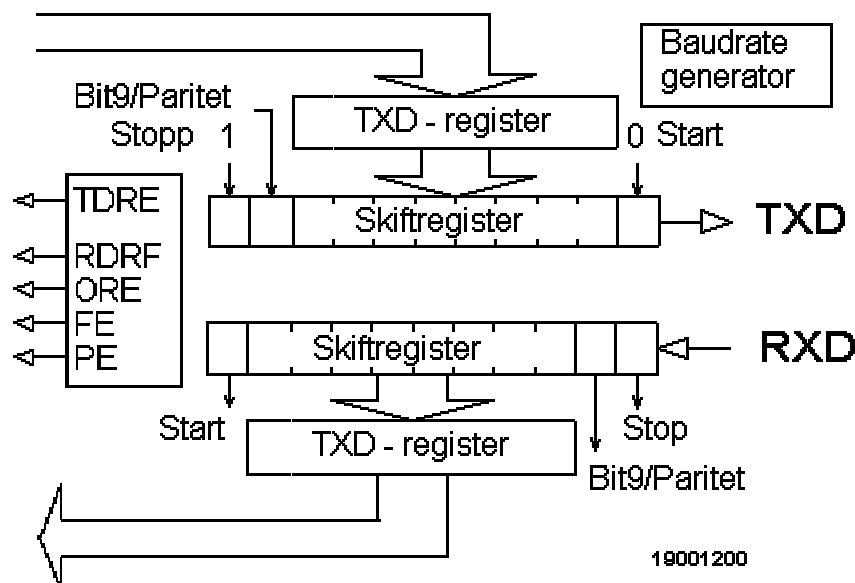
Även om sändare och mottagare är överens om vilken dataöverföringshastighet som ska gälla kan det finnas tillfällen då mottagaren tillfälligtvis önskar hejda dataflödet. Det kan tex. gälla en skrivare som normalt hinner skriva tecknen i precis den takt de anländer, men kan behöva mer tid på sig var gång skrivarhuvudet ska positioneras från slutet av en rad till början av nästa, eller när en sida är fullskriven och ett nytt pappersark ska matas in från en arkmatare. Det kan gälla en dator som ibland behöver få "arbetsro" medan den arbetar med någon speciellt krävande uppgift, etc. etc.

En vätskebehållare kan ses som en analogi till en databuffert.
Dataflödet startas när nivån ligger under XON och hejdas vid nivån XOFF.



Dataöverföringen kan emellertid inte stoppas omedelbart. I de fall då en mottagande enhet inte kan garantera att alltid ta emot med den överenskomma överföringshastigheten, måste den därför förses med ett buffertminne. Med två speciella tecken XON/XOFF (sändning på/ sändning av) kan den mottagande enheten meddela sändaren att bufferten håller på att bli full. Detta sker med hjälp av en extra serieförbindelse i den motsatta riktningen (det krävs således full duplex). Som en analogi till databufferten visas i figuren en vätskebehållare som kan fyllas på från en kran, och tömmas från botten. Två nivåvakter XON och XOFF registrerar vätskenivån. När vätskan nått upp till nivån XOFF ska påfyllningen stoppas. XOFF sitter monterad betryggande långt från behållarens kant, så att det inte ska vara någon risk att behållaren blir full under den tid det kan ta att stoppa påfyllningen. När behållaren tömts förbi XON kan påfyllningen av behållaren återupptas. Som tecknen XON/XOFF brukar man använda 17/19 (DC1/DC3, Crtl-Q/Ctrl-S, 00010001/00010011).

UART - kretsen för asynkron seriekommunikation



Blockschema över en UART (Universal Asynchronous Reciever/ Transmitter).

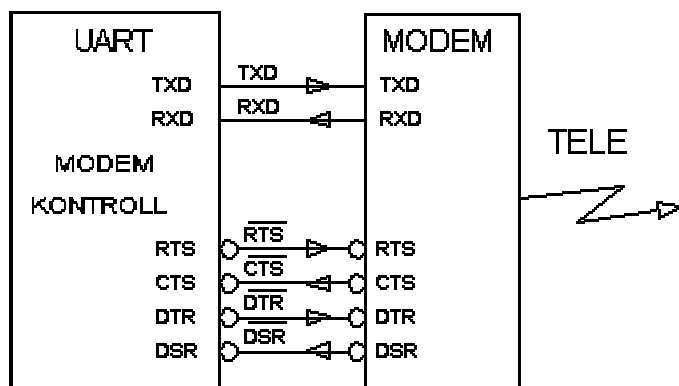
Asynkron seriekommunikation är mycket vanligt förekommande i mikrodatorsystem. Som vi sett är överföringsformatet komplicerat och förekommer i många varianter. Lyckligtvis finns det färdiga standardkretsar som kan hantera hela denna kommunikationsprocess. De kallas för **UART (Universal Asynchronous Receiver/Transmitter)**. En ofta använd sådan krets har beteckningen 8250. Det är även vanligt att en UART finns med bland de inbyggda modulerna i mikrostyrciklar. Om man har en mikrostyrcikla med en inbyggd "UART", så ställs den in och styrs med några speciella funktionsregister, SFR. Från SFR-registren kan man även avläsa UART:ens status-signaler, och i allmänhet låta någon/några status-signaler generera interrupt.

Innan man påbörjar datakommunikationen måste UART:en ställas in för den aktuella hastigheten och det aktuella överföringsformatet. Det vill säga rätt Baudrate, antal databitar, ev paritetskontroll och antalet stoppbitar måste ställas in. För att sända en byte kontrollerar man först med statusbiten **TDRE (Transmitter Data Register Empty)** om TXD-registret är tomt. När så är fallet kan man skriva en byte till TXD-registret. UART:en kommer då att automatiskt överföra denna byte till skiftregistret så fort som möjligt, det vill säga när detta är färdigt med den föregående byten. Med hjälp av TDRE-biten kan man sedan hela tiden hålla reda på när UART:en kan tillföras en ny byte för sändning.

Om man programmerar att TDRE genererar interrupt, och denna interruptsbrutin hämtar de bytes som ska sändas från minnet, kan sändningen fortgå automatiskt i "bakgrunden" utan att nämvärt belasta huvudprogrammet. Vid mottagningen av en byte kontrollerar UART:en hela processen. Om startbiten är kortare än halva bit-tiden räknas detta som en störning av linjen och ignoreras. Om stoppbiten är för kort markerar UART:en detta med att sätta biten **FE (Framing Error)**. UART:en beräknar även pariteten och markerar eventuella paritetsfel genom att sätta biten **PE (Parity Error)**. Med biten **RDRF (Receiver Data Register Full)** anges när en mottagen byte kan hämtas från RXD-registret. När man hämtar byten 0-ställs RDRF automatiskt. Om man inte hunnit hämta en Byte från RXD-registret *innan* nästa byte mottagits av skiftregistret (och måste överföras till RXD-registret), kommer byten att gå förlorad. Biten **ORE (OverRun Error)** anger om detta hänt. Om man programmerar RDRF att generera interrupt, och denna interruptsbrutin lämnar alla mottagna bytes till en kö/teckenbuffert (en programkonstruktion), kommer mottagandet av bytes att fortgå i "bakgrunden" och belasta huvudprogrammet obetydligt.

I tillämpningar när man inte behöver sända och mottaga tecken samtidigt är det vanligt att man använder en enklare (och billigare) mikrostyrcikla som saknar UART. Då programmerar man själv seriekommunikationen "bit för bit". Med hjälp av handskakningssignaler kan man vid behov "stänga av" kommunikationen för att ge processorn tid med tex beräkningar och datainsamling.

Modem styrsignaler



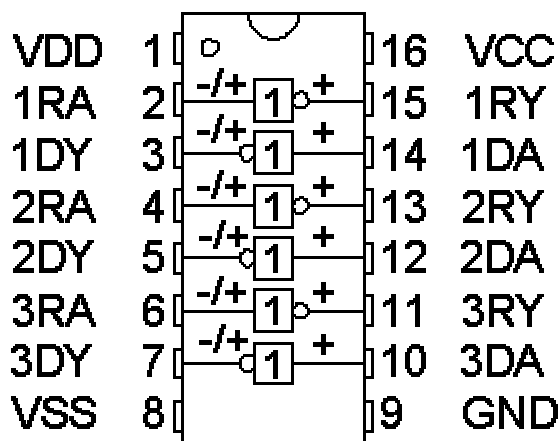
Anslutning av en asynkron serieport till ett modem för kommunikation över telenätet.

Vid seriell dataöverföring över mycket långa avstånd kan man använda sig av telenätet. Eftersom telenätet inte kan överföra logiknivåer (= likspänningar) omformas dessa till tonsignaler med hjälp av **Modem** (**MO**dular **DE**Modulator). Exempelvis använder asynkrona modem för 1200 Baud tonerna ("0"/"1") 1180/980 Hz för det uppringande, och 1850/1650 för det svarande modemmet. För att kunna styra och kontrollera uppkopplingen av en uppringd förbindelse är modemerna försedda med handskakningssignaler. (Begreppet handskakning har förklarats tidigare). I princip ska varje åtgärd som endera sidan företar sig för att etablera förbindelsen kvitteras av den andra sidan.

Det är viktigt att känna till dessa styrsignaler eftersom det har blivit så att de fått en mycket vidare användning än bara inom telenätet.

I figuren visas de vanligaste anslutningarna till ett modem. Förutom de seriella datasignalerna RXD och TXD, förekommer det två nivåer av handskakningssignaler. Signalparet DTR (**D**ata **T**erminal **R**eady, UART klar) och DSR (**D**ata **S**et **R**eady, modem klar) är den överordnade handskakningsnivån, medan RTS (**R**equest **T**o **S**end, sändningsbegäran från UART) och CTS (**C**lear **T**o **S**end, datakanal klar från modemmet) är en underordnad handskakningsnivå.

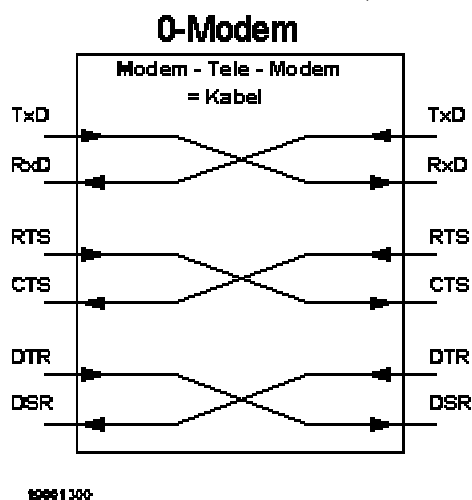
Seriekommunikationsstandarden kallas för RS232. Datasignalerna (RXD, TXD) tillämpar aktivt hög logik medan styrsignalerna (RTS, CTS, DTR, DSR) tillämpar aktivt låg logik. Signalnivåerna är inte de vanliga logiknivåerna, utan modemsignaler ska växla mellan ± 12 V. "0"/"1" svarar mot $+12/-12$ V. Den standard som beskriver spänningsnivåerna kallas för V24. Kretsen 1406V24 kan användas för att omvandla mellan dessa signalnivåer. Den räcker till en modemförbindelse med samtliga ovan beskrivna signaler. (Matningsspänningar: VCC=+5V, VDD=+12V, VSS=-12V, GND).



Förr var det vanligt processorn och övriga kretsar också behövde dessa tre matningsspänningar, idag klarar sig så gott som alla vanliga kretsar med enkel matningsspänning (+3 ... +5 V). Den som vill använda seriekommunikation måste således för detta investera i dyra spänningsaggregat med flera utgångsspänningar. Ett alternativ är att använda kommunikationskretsar med inbyggda spänningsomvandlare. Kretsen MAX232 har inbyggd likspänningsomvandlare, och behöver därmed bara en matningsspänning, +5 V.

0-Modem (nollmodem)

Genom att olika länders teleadministrationer enats om en modemstandard har man "på köpet" fått en möjlighet att koppla samman datorer och datorutrustningar av olika fabrikat. Alla datorutrustningar som kan anslutas till telenätet kan i princip även anslutas direkt till varandra. (Detta oavsett om fabrikanten avsett detta eller ej!)



Ett sk. 0-modem, dvs en kabel som är avsedd att ersätta en modemförbindelse.

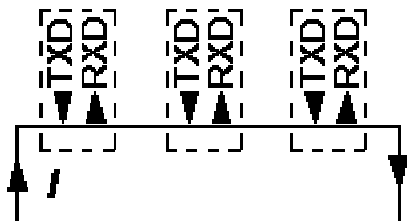
För att ansluta två utrustningar som har modemkontakter till varandra måste man tillverka ett så kallat 0-modem. Det är en kabel där signalparen RXD - TXD, DTR - DSR, och RTS - CTS korsats. (Den ena enheten är ju sändare för den andra och vice versa). Med ett 0-modem får man tillgång till handskakningssignalerna och kan använda dessa till flödeskontroll.

Om man inte ansluter handskakningssignalerna kan flödeskontrollen ske med kommunikationssignaler, XON/XOFF, eller så måste utrustningen ha så höga prestanda att man klarar man sig utan flödeskontroll.

Det är få tillämpningar, förutom modem, som behöver handskakningssignaler i två nivåer. Vanligen används signalparet RTS - CTS till den parallella handskakningen. Signalparet DTR - DSR brukar ges en enklare hantering (eller ibland uteslutas).

Ledningen DTR brukar ofta anslutas till respektive datautrustnings +12 V matningsspänning (eftersom signalen är aktiv låg svarar +12V mot "0"). Med DTR kan då utrustningarna upptäcka enklare fel som att man "glömt" att slå på matningsspänning till den andra enheten, eller att man "glömt" att ansluta 0-modem-kabeln.

Strömslinga - kommunikation mellan flera enheter



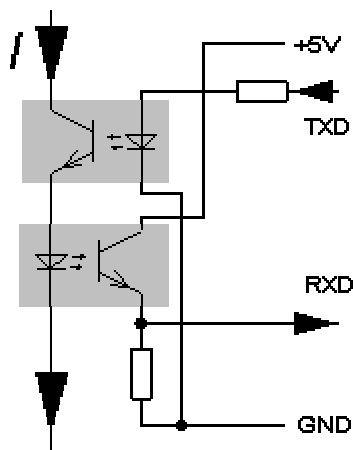
Strömslinga. De kommunicerande enheterna kopplas i serie med varandra som i en "julgransbelysning".

Vid asynkron seriekommunikation behövs det i princip direkta ledningar mellan alla de enheter som ska kunna kommunicera med varandra. Ska "alla" kunna kommunicera med "alla" blir ledningsnätet omfattande.

Ofta är emellertid en av enheterna "huvudenhet" och de övriga enheterna "slavenheter". Alla slavenheter har då ledningar till huvudenheten, men inga ledningar mellan varandra. Huvudenheten fungerar som en "kommunikationsväxel", som vid behov "kopplar ihop" enheterna med varandra.

En gammal teknik för att enkelt knyta samman enheter som använder asynkron seriekommunikation är strömslingan. Alla enheter är då anslutna till en gemensam "seriekanal" och kan avlyssna kommunikationen, men endast en enhet åt gången får sända på denna kanal (dvs. halv duplex).

En fördel med strömslingan var att det enkelt gick att ansluta ytterligare enheter, i princip kunden man bryta upp slingan var som helst för att placera in en ny enhet. Strömslingan kan jämföras med en julgransbelysning med seriekopplade glödlampor. Varje enhet symboliseras av en glödlampa. Man sänder en "nolla" från någon glödlampa genom att skruva ur den så att den slocknar, då slocknar alla andra (seriekopplade) glödlampor och kan på så sätt sägas ha mottagit "nollan".



Strömslinge-enhet bestående av två optokopplare. *I* är strömmen i strömslingan som bär informationen mellan enheterna. TXD och RXD är enhetens kommunikationssignaler.

Strömslingan bygger på optokopplare, en lysdiod och en fototransistor i samma kapsel. Får lysdioden ström så lyser den på transistor så att denna släpper igenom ström. Om lysdioden inte lyser så spärrar transistor strömmens väg.

Varje strömslinge enhet innehåller två optokopplare, en sändare och en mottagare. Den övre optokopplaren styrs av signalen TXD. När TXD är hög (+5V) lyser lysdioden och transistor släpper fram strömmen genom strömslingan. Lysdioden i den undre optokopplaren lyser då på sin transistor och det leder i sin tur till att signalen RXD också är hög (+5V). Detta är utgångsläget.

Om nu någon strömslinge enhet, valfritt vilken, ändrar sin TXD till låg (0V) så bryts strömmen i slingan och alla enheter får RXD låg (0V). Varje strömslinge enhet kan på detta sätt sända information till de andra enheterna.

Strömmen till strömslingan tas från en likspänningskälla U . Man kan räkna med att spänningsfallet i varje strömslinge enhet uppgår till c:a 3 V, och man brukar reducera slingströmmen I till 20 mA med en serieresistor (man använder Ohms lag för att beräkna denna $U - n \cdot 3 - I \cdot R = 0$ där n är antalet strömslinge enheter). Det som begränsar antalet strömslinge enheter är att U inte får ha ett högre värde än vad transistorerna i strömslinge enheterna tål.

Strömslingan används sällan idag. Principen att låta den ena logiska nivån vara dominant (i strömslingans fall "0", = avbrott) används däremot av så gott som alla dagens kommunikationsbussar (tex. CAN).

Adressering med en "nionde" databit

Det finns ett speciellt överföringsformat som underlättar användningen av en gemensam "seriekanal". En UART kan ofta programmeras att byta ut sin paritetsbit mot en nionde databit. Denna extra databit kan användas av huvudenheten för att styra kommunikationen.

Antag att vi har en huvudenhet och ett antal slavenheter längs en strömslinga. Vid normal dataöverföring är den nionde biten i alla byte's som sänds 0:ställd. Slavenheternas UART kan programmeras på två sätt. Antingen gör den bara interrupt när den mottagit en byte's som har den nionde biten 1:ställd, eller så gör den interrupt för alla byte's den mottager. Från början är slavenheterna programmerade att bara göra interrupt när den *nionde biten* är 1:ställd. De ignorerar då all normal (nionde biten 0:ställd) dataöverföring som kan förekomma längs slingan. Om huvudenheten sänder ut en byte med den nionde biten 1:ställd, kommer denna att mottagas av *alla* slavenheter. Den mottagna bytens värde ska då tolkas som adressen till en av slavenheterna, den som huvudenheten vill kommunicera med. Den utpekade slavenheten programmerar då om sin UART att ge interrupt för *varje* mottagen byte, och kommunikationen med huvudenheten kan börja.

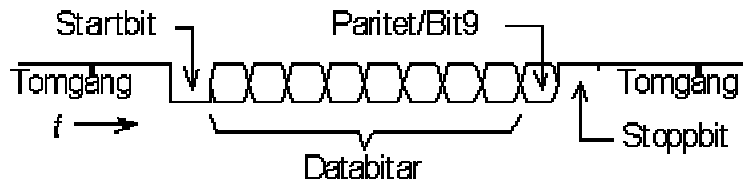
Synkron seriekommunikation

Om data sänds seriellt som en enda lång ström av databitar, utan startbitar och stoppbitar, talar man om synkron seriekommunikation. Sändare och mottagare måste arbeta i takt med varandra, och behöver därför ha speciella synkroniseringskretsar. Enklast kan detta lösas om, förutom datasignalen, även klockpulserna till synkroniseringskretsarna överförs. I annat fall måste sändare och mottagare ha mycket stabila oscillatorkretsar. Mottagaren bör även ha möjlighet att finjustera sin oscillatorfrekvens med hjälp av den mottagna datasignalen (detta kallas för faslåsnings).

För att kunna markera var ett datablock börjar (och ett annat slutar) i bitströmmen, används en speciell synkroniseringssekvens, nämligen 8 ettor i en följd. Nu kan naturligtvis ibland datasignalen själv innehålla 8 ettor i följd. För att detta inte felaktigt ska tolkas som slutet av eller början på ett block, ser man till att det vid sändningen av data, skjuts emellan en nolla så fort det sänts sju ettor i en följd. Sändningen av synkroniseringstecknet sker däremot med 8 ettor i en följd. Mottagaren känner till detta, och tar därför alltid bort en nolla som följt på sju ettor. Synkroniseringssekvensen förblir därigenom unik.

Det går att uppnå högre överföringshastigheter med synkron seriekommunikation än med asynkron, eftersom antalet "onödiga" bitar är mindre. Vid asynkron överföring krävs det minst en startbit och stoppbit per 8 databitar, dvs $2/8 = 25\%$ "onödiga" bitar. Om man tex istället överför 100 bytes med synkron överföring, behöver man totalt bara ca 10 onödiga bitar (1 synkroniseringssekvens + ett okänt antal inskjutna 0:or, totalt ca 10 st bitar), dvs $10/800$ ca 1 % onödiga bitar.

Seriekommunikation `getchar()` `putchar()`



Teckenformatet vid asynkron seriell kommunikation.

ANSI-C har funktionen `getchar()` för att hämta ett tecken från tangentbordet till ett exekverande program, och funktionen `putchar()` för att leverera ett tecken från programmet till bildskärmen. Formatterad inmatning och utmatning bygger sedan vidare på dessa grundläggande funktioner.



Tangentbord (parallellt) och display (seriell) till inbyggnadsprocessorer.

I en typisk tillämpning med en inbyggnadsprocessor så har vi varken tangentbord eller bildskärm, utan i stället ofta någon typ av teckenfönster och någon enklare knappsats. Har processorn seriekommunikation kan den anslutas till en dataterminal. Även enkla inbyggnadsprocessorer har ofta denna anslutningsmöjlighet, åtminstone för serviceändamål.

Eftersom förutsättningarna varierar så mycket måste man själv skriva in- och utmatningsfunktioner speciellt för varje tillämpning. En bra idé är att snegla på hur motsvarande funktioner brukar lösas i ANSI-C!

Seriekommunikation med PIC 16F84 (utan UART)

Vi börjar med att titta på hur man löser seriekommunikation för PIC16F84 som *inte* har någon inbyggd enhet för seriekommunikation.

Ofta har man inget behov av att sända eller ta emot tecken samtidigt som programmet ska göra något annat, och för de fallen blir rutinerna för seriekommunikation enkla och då kan man mycket väl skriva programkod som tar hand om detta på "bitnivå".

Funktionen `initserial()`

När man programmerar seriekommunikationen själv är så gott som alla portpinnar användbara till detta, man kan dessutom programmera många seriekanaler. Vi använder här RB0 som serieutgång och RB1 som serieingång. Detta måste ställas in i TRISB-registret. Från början, och mellan tecknen ska serieutgången vara hög, vi skriver därför en initialiseringsfunktion. Denna ska anropas *en gång* i början av huvudprogrammet innan man använder övriga funktioner.

```
void initserial( void ) /* initialise serialcom port */
{
    PORTB.0 = 1;
    TRISB.0 = 0;
    TRISB.1 = 1;
}
```

Funktionen `putchar ( )` 9600 Baud @ 4 MHz (Cc5x- kompilatorn)

RRF Rotate Right f through Carry



PIC-processorns instruktion RRF används vid seriekommunikation.

```
bit putchar( char d_out ) /* sends one char */
{
    char bitCount, ti;
    /* One start bit, one stop bit, 8 data bit, no parity = 10 bit. */
    /* Baudrate: 9600 baud => 104.167 usec. per bit. */
    if(d_out == '\0') return 0; /* refuse to send "end of string" */
    PORTB.0 = 0; /* set startbit */
    for ( bitCount = 10; bitCount > 0 ; bitCount-- )
    {
        /* delay one bit 104 usec at 4 MHz (5+30*3-1+1+9=104) */
        ti = 30; do ; while( --ti > 0); nop();
        Carry = 1; /* stopbit */
        d_out = rr( d_out ); /* Rotate Right through Carry */
        PORTB.0 = Carry;
    }
    return 1; /* all done */
}
```

Funktionen är av typen Bit och returnerar 1 när den sänder ett tecken och 0 om den inte sänder tecknet (i så fall har det rört sig om '\0' -tecknet, det som avslutar teckensträngar och som inte ska skrivas ut).

Kärnan i rutinen är rotationsinstruktionen RRF som man når med den interna (= inbyggd i Cc5x) C-funktionen `d_out = rr( d_out );`.

Bitvariabeln `Carry` är fördefinierad i Cc5x. Man kan lagra "1:or" i `Carry` som skiftas in i registret, och i `Carry` kan man även läsa de bitar som skiftas ut.

Funktionen `getchar( )` 9600 Baud @ 4 MHz (Cc5x- kompilatorn)

```
char getchar( void ) /* recieves one char */
{
    /* One start bit, one stop bit, 8 data bit, no parity = 10 bit. */
    /* Baudrate: 9600 baud => 104.167 usec. per bit. */
    char d_in, bitCount, ti;
    while( PORTB.1 == 1 ) /* wait for startbit */ ;
    /* delay 1,5 bit 156 usec at 4 MHz ( 5+47*3-1+2+9=156) */
    ti = 47; do ; while( --ti > 0); nop2();
    for( bitCount = 8; bitCount > 0 ; bitCount--)
    {
        Carry = PORTB.1;
        d_in = rr( d_in); /* rotate carry */
        /* delay one bit 104 usec at 4 MHz (5+30*3-1+1+9=104) */
        ti = 30; do ; while( --ti > 0); nop();
    }
    return d_in;
}
```

Funktionen är av typen `char` och returvärdet är det mottagna tecknet. Funktionen börjar med att vänta på startbiten, så har man väl anropat funktionen kan programmet inte göra något annat än att vänta! (Här kan man således fastna).

Detta kan man undvika genom att *inte* anropa funktionen förrän man på *annat sätt* i programmet upptäckt att startbiten påbörjats.

(ex. `if( PORTB.1 == 0 ) ch = getchar();`).

Vill man använda seriekommunikation till **PIC16F84** kan man skriva in dessa funktioner i sin main-fil. Funktionerna finns också samlade i en funktionsfil

`seriF84.c`.

I stället för att skriva in funktionerna kan man inkludera dem med kommandot

`#include "seriF84.c"`.

Funktionsprototyperna finns i filen `seriF84.h`.

Lösta programexempel med PIC16F84

Seriekommunikation programmet echo84.c

Ett enkelt testprogram för att kontrollera om en serieförbindelse fungerar är echo84.c. Från en terminal, som *inte* är inställd för lokalt eko, skriver man tecken som PIC-processorn ska "eka" tillbaka till terminalen. Om detta *inte* sker, har man problem med överföringen.

Vid radslut kan två tecken behöva sändas till terminalen (LF line feed och CR carriage return, radframmatning och återgång till radens början). Det ombesörjes på raden `if( c == '\r') { putchar('\r'); putchar('\n'); }`. (detta är en "inställningsfråga", en del terminalprogram kan själva lägga till det extra tecknet, men vi låter i det följande PIC-processorn göra detta).

En rolig variant på programmet är som "Kryptomaskin". Genom att inte "eka" tillbaka rätt tecken, utan exempelvis `putchar( c + 1 );` blir texten förvrängd.

```
/* echo84.c  echo characters, serial input/output ( for 16F84) */

/*
SERIAL COMMUNICATION (RS232)
=====
One start bit, one stop bit, 8 data bit, no parity = 10 bit.
Baudrate:
9600 baud = 104.167 usec. per bit.
ser_in RB1 ser_out RB0
*/

#include "16F84.h"

/* Circuitprogrammer Configuration: FOSC=XT, WDTE=off, PWRT=on,
CP=off
*/

#pragma config |= 0x3ff1

bit putchar( char ); /* function prototype */
char getchar( void ); /* function prototype */

void main( void)
{
    char c;
    PORTB = 0b00000001;
    TRISB = 0b11111110;
    while( 1)
    {
        c = getchar( ); /* input 1 character */
        if( c == '\r')
        {
            putchar('\r');
            putchar('\n');
        } /* "Line Feed" and "Carriage Return" */
        else putchar(c); /* echo the character */
    }
}
```



```

/*
  Functions
  =====
*/

bit putchar( char d_out ) /* sends one char */
{
  char bitCount, ti;
  /* One start bit, one stop bit, 8 data bit, no parity = 10 bit. */
  /* Baudrate: 9600 baud = 104.167 usec. per bit. */
  if(d_out == '\0') return 0; /* refuse to send "end of string" */
  PORTB.0 = 0; /* set startbit */
  for ( bitCount = 10; bitCount > 0 ; bitCount-- )
  {
    /* delay one bit 104 usec at 4 MHz (5+30*3-1+1+9=104) */
    ti = 30; do ; while( --ti < 0); nop();
    Carry = 1; /* stopbit */
    d_out = rr( d_out ); /* Rotate Right through Carry */
    PORTB.0 = Carry;
  }
  return 1; /* all done */
}

char getchar( void ) /* recieves one char */
{
  /* One start bit, one stop bit, 8 data bit, no parity = 10 bit. */
  /* Baudrate: 9600 baud = 104.167 usec. per bit. */
  char d_in, bitCount, ti;
  while( PORTB.1 == 1 ) /* wait for startbit */ ;
  /* delay 1,5 bit 156 usec at 4 MHz ( 5+47*3-1+2+9=156) */
  ti = 47; do ; while( --ti < 0); nop2();
  for( bitCount = 8; bitCount > 0 ; bitCount--)
  {
    Carry = PORTB.1;
    d_in = rr( d_in); /* rotate carry */
    /* delay one bit 104 usec at 4 MHz (5+30*3-1+1+9=104) */
    ti = 30; do ; while( --ti < 0); nop();
  }
  return d_in;
}

```

PIC-processorer med inbyggd seriekommunikationskrets (tex. 16F628)

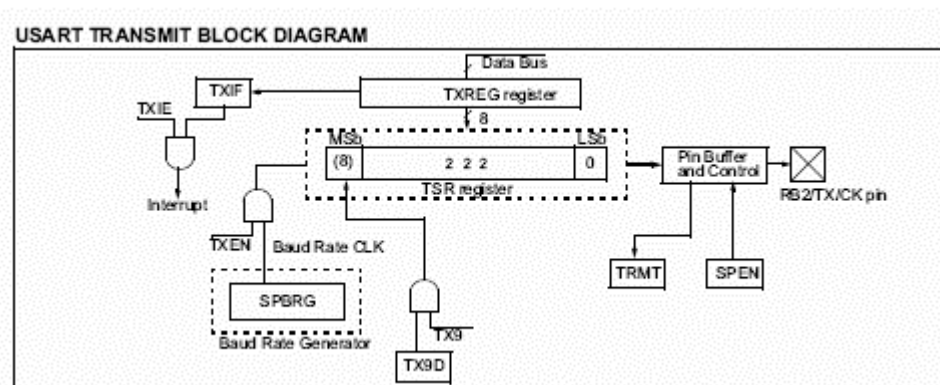
USART

Många PIC-processorer innehåller en inbyggd seriekommunikationsenhet, **USART** (**U**niversal **S**ynchronous or **A**synchronous **R**eciever and **T**ransmitter). Som namnet anger är denna enhet användbar *både* för synkron och asynkron seriekommunikation, men vi kommer bara utnyttja asynkron seriekommunikation.

USART består av tre delar. **SPBRG** (**S**erialunit **P**rogramable **B**aud**R**ate**G**enerator) är en programmerbar Baudgenerator för överföringshastigheten.

USART Transmitter och **USART Reciever** som är sändarenhet respektive mottagarenhet.

USART Transmitter

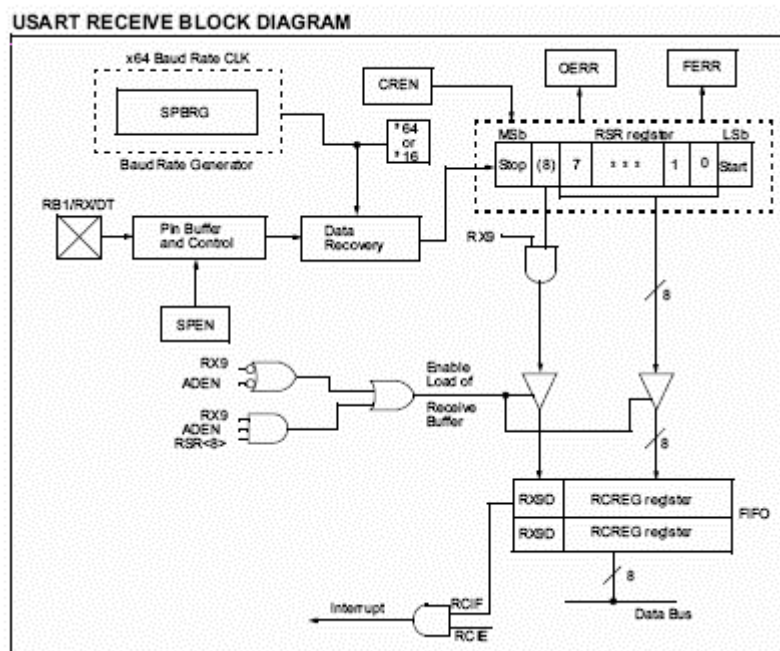


För att sända ett tecken räcker det med att lägga det i TXREG-registret. När sändarregistret TSR är "redo" kopieras tecknet över till detta och skiftas ut seriellt på pinnen RB2/TX/CK (pinne 8). Om man har ytterligare ett tecken att sända kan man nu placera detta i "väntkö" i TXREG. Så fort TSR är tomt laddas nästa tecken in från TXREG automatiskt till TSR.

Ett program kan på detta sätt lämna två tecken och behöver sedan inte "bry sig om" sändningen förrän det är dax för ytterligare tecken.

I blockdiagrammet visas flaggan TRMT (**T**ransmitter**R**egister **eM**p**T**y) som anger om sändarregistret TSR är upptaget eller ej. Flaggan kan läsas av programmet (eller generera interrupt) så att man på så sätt kan se till att tecken skickas kontinuerligt. I blockdiagrammet finns även en flagga TX9D för kommunikation med nio bitar, tex. om man använder paritetsbit, eller adresseringsbit vid kommunikation med flera processorer över samma ledning. (Se tidigare avsnitt om datakommunikation).

USART Reciever



Tecken inkommer "bit för bit" från pinnen RB1/RX/DT (pinne 7) till mottagarregistret RSR. När mottagningen av ett tecken är klart förs det över till RCR som är en så kallad FIFO-buffert. Denna buffert rymmer två tecken, **FIFO** (**F**irst **I**n **F**irst **O**ut) innebär att de kan läsas i samma ordning som de kom. Bufferten medför att ett program kan låta bli att bevaka mottagarregistret under den tid det tar att mottaga två tecken. *Om ett tredje tecken hinner inkomma innan programmet läst de tidigare tecknen "låser sig" dock mottagarenheten.*

I blockdiagrammet finns en flagga **RCIF** som anger om det finns tecken i bufferten eller ej (kan även generera interrupt). Denna flagga "0"-ställs *automatiskt* när man läst så att bufferten blivit tom, ett/två tecken.

Övriga flaggor **OERR**, **FERR** varnar för felaktigt mottagna tecken.

Blocken **Pin Buffer and Control** och **Data Recovery** filtrerar bort störningar från den mottagna seriesignalen. Dessa block innebär att det blir mycket säkrare kommunikation om man använder den inbyggda seriekommunikationsenheten än om man programmerar "bit för bit".

Seriekommunikationsfunktioner i filen seriF62x.c

Inställningsfunktionen `initserial()`

Medan funktionen `initserial()` för kretsen 16F84 är mycket enkel, i stort sätt bara inställning av portpinnarna, är det betydligt mer att tänka på och ta hänsyn till när man ska använda seriekommunikationsenheten.

```
void initserial( void ) /* initialise serialcom port */
{
    SPEN = 1;
    BRGH = 1; /* Async high speed */
    TXEN = 1; /* transmit enable */
    SPBRG = 26-1; /* 9600 Baud @ 4 MHz-clockfrequency */
    CREN = 1; /* Continuous receive */
    RX9 = 0; /* 8 bit reception */
    TRISB.2 = 0; /* TX is output */
    TRISB.1 = 1; /* RX is input */
}
```

Baudrate ställs in med flaggan `BRGH` och registret `SPBRG`. I processorns datablad finns formler för att beräkna lämpliga värden. För de vanligaste Baudrate-värdena och de vanligaste klock-kristallerna finns det dessutom färdiga tabeller.

Flaggan `SPEN` kopplar in hela seriekommunikationsenheten, och flaggan `TXEN` kopplar in Baudrategeneratoren till sändarenheten.

Serie-ingång och utgång måste ställas in med `TRISB`-registret.

Funktionen `initserial()` ska anropas en gång i början av programmet innan man gör något annat med seriekommunikationsenheten.

Funktionen `putchar()`

```
bit putchar( char d_out ) /* sends one char */
{
    if(d_out == '\0') return 0;
    /* refuse to send 0 "end of string" */
    while (!TXIF) ;
    /* wait until previous character transmitted */
    TXREG = d_out;
    return 1; /* done */
}
```

Funktionen har en bit som returvärde "1"/"0". Den returnerar "0" om tecknet den skulle sända är "strängslutstecknet". Detta är ett programmeringsknep som gör det enklare att sända en sträng "från början till slut". Mer om detta senare.

Funktionen `getchar()`

```
char getchar( void )      /* recieves one char */
{
    char d_in;
    while ( RCIF == 0 ) ; /* wait for char */
    d_in = RCREG;
    return d_in;
}
```

Varning! Mottagarenheten kan låsa sig! **Funktionen `OverrunRecover()`**

Som tidigare nämnts så måste man (programmet) läsa mottagarenheten innan den hunnit mottaga tre tecken - annars låser den sig!

När man ansluter seriekontakten kanske man "darrar" på handen på sådant sätt att "kontaktstudsarna" blir till många mottagna tecken. Om mottagarenheten då "låser sig" är detta naturligtvis ett mycket svårt/omöjligt "programfel" att hitta!

Lösningen är en upplåsningsrutin att kunna ta till vid behov.

Man anropar upplåsningsfunktionen direkt innan man förväntar sig inmatning via serieporten. Om serieporten inte låst sig gör funktionen i varje fall ingen skada!

```
/* If two (or more) char's are recieved,                */
/* but not read, the reciever will be locked!           */
/* To unlock the reciever, run function OverrunRecover() */

void OverrunRecover(void)
{
    char trash;
    trash = RCREG; /* the two char's that locked the reciever */
    trash = RCREG; /* are read and ignored                    */
    CREN = 0;      /* the unlock procedure ...                */
    CREN = 1;
}
```

Dessa funktioner finns i filen `seriF62x.c`.

Funktionsprototyperna finns i filen `seriF62x.h`.

Lösta programexempel med PIC16F628

Seriekommunikation programmet **echo62x.c**

Här följer testprogrammet `echo62.c` som man använder med PIC16F628 för att kontrollera att man har en fungerande uppkoppling mellan PC och PIC-processor.

```
/* echo62x.c  echo characters, serial input/output,  */

#include "16F628.h"
#pragma config _|= 0x3ff0

/* Function prototypes */
bit putchar( char ); /* function from seriF62x.c */
char getchar( void ); /* function from seriF62x.c */
void initserial( void ); /* function from seriF62x.c */
void OverrunRecover(void); /* function from seriF62x.c */

#include "seriF62x.c"

void main( void)
{
    char c;
    initserial();
    OverrunRecover();
    while( 1)
    {
        c = getchar( ); /* input 1 character */
        if( c == '\r')
        {
            putchar('\r');
            putchar('\n');
        }
        else putchar(c); /* "Line Feed" and "Carrige Return" */
    }
}
```

Programmet finns i filen `echo62x.c`. Funktionerna finns i filen `seriF62x.c`. Funktionsprototyperna finns i filen `seriF62x.h`.

Lösta programexempel med PIC16F628

Seriekommunikation "Hello World"

hello62x.c

Alltsedan första utgåvan av Kernighan/Rithie's lärobok i C, brukar man som enkelt testprogram skriva ut texten "hello world" på bildskärmen. Det får därför bli vår första utskrift.

Kompilatorn **Cc5x** tillåter *inte* strängkonstanter. Man kan således *inte* definiera den mening man tänker skriva ut som med standard ANSI-C:

```
text1[]="hello world\r\n";
```

PIC-processorerne lagrar strängkonstanter med bokstäverna inbakade i en följd av instruktioner (en tabell).

Man hämtar en bokstav genom att först ladda W-registret med bokstavens ordningsnummer och därefter göra ett subrutinanrop, tex **CALL text1**. Tabellen **text1** börjar med instruktionen **ADDWF** som innebär att hoppet går vidare till önskat ordningsnummer i tabellen (enligt talet i W-registret). Där finner man instruktionen **RETLW** som innebär återhopp, men nu med den önskade bokstaven i W-registret.

text1	addwf	PCL,F	; jump indirect w
	retlw	0x48	;'H'
	retlw	0x65	;'e'
	retlw	0x6c	;'l'
	retlw	0x6c	;'l'
	retlw	0x6f	;'o'

- - - - -

Assemblerinstruktioner som definierar textsträngen "Hello...".

Det finns inte något självklart sätt att göra detta med C. Kompilatorn **Cc5x** har infört en funktion **skip(x)**; som innebär att man hoppar över x st C-instruktioner (funktionen är inbyggd i Cc5x). I stället för strängkonstanter som **text1[]**, använder man funktioner som **char text1(char x);**.

```
char text1( char x)
{
    skip(x);
    #pragma return[] = "hello world" '\r' '\n' '\0'
}
```

Cc5x C-funktion som definierar textsträngen "hello world".

Förutom den vanliga return-satsen för ett returvärde har **Cc5x** infört **#pragma return[]** som ger möjlighet att skriva flera bokstäver i följd som vanliga meningar. Detta skrivsätt gör programmet tydligare. Strängen avslutas med tecknet **'\0'** som är C:s 0-tecken. (Tecknen **'\r'** och **'\n'** ger en radmatning).

Strängen skrivs ut med raden `for(i = 0; putchar( text1(i)); i++) ;`. Vi drar här nytta av att `putchar( )` skrivits så att den *inte* kan sända `'\0'`-tecken utan då returnerar talet 0 i stället, vilket direkt avslutar for-slingan. På detta sätt kan man skriva ut strängar utan att behöva hålla reda på hur långa de är.

Seriekommunikation "Hello world" (Cc5x-kompilatorn)

```
/* hello62x.c "Hello World!" program for 16F628 */
/* 9600 baud */

#include "16f628.h"
#pragma config |= 0x3ff0
/* function prototypes */
void initserial( void );      /* from seriF62x.c */
bit putchar( char );         /* from seriF62x.c */
void delay( char );          /* from delays.c */
char text1( char );

#include "seriF62x.c"

void main(void)
{
    char i;
    /* Initialise serial port */
    initserial();
    putchar('\r'); putchar('\n');

    /* print textstring text1 */
    for(i = 0; putchar( text1(i)) ; i++) ;
    delay(10);
}

/* String to write */
char text1( char x)
{
    skip(x);
    #pragma return[] = "hello, world!" '\0'
}
```

Programmet finns i filen `hello62x.c`. Funktionerna finns i filerna `seriF62x.c`. och `delays.c`.

Funktionsprototyperna finns i filen `seriF62x.h`.

Motsvarande programfil för 16F84 är `hello84.c` med funktionerna i filen `seriF84.c` och funktionsprototyperna finns i filen `seriF84.h`.

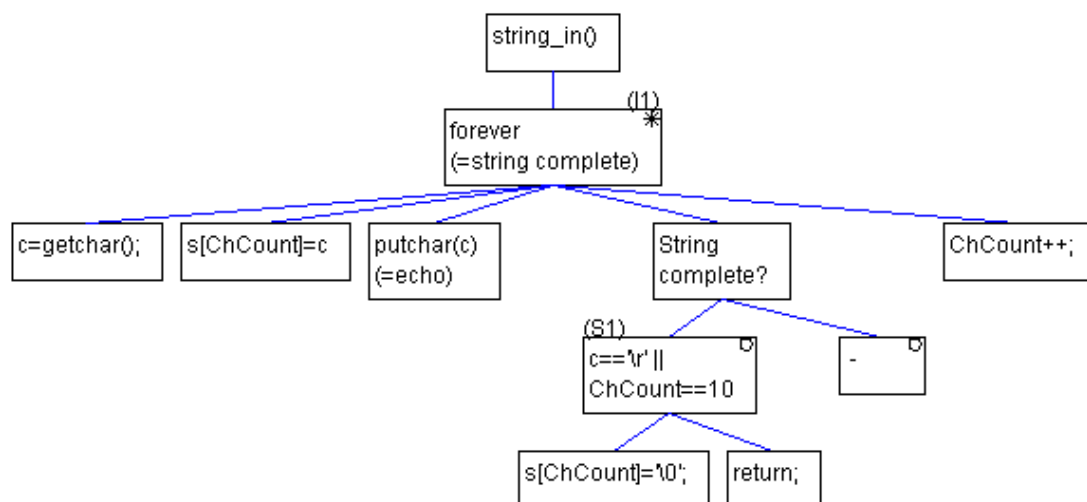
Lösta programexempel med PIC16F628

Seriekommunikation, jämför strängar comp62x.c

Mata in en sträng

Användaren matar in tecken som placeras i en inmatningsbuffer `s[]` som rymmer 11 tecken. Med satsen `if( (c == '\r') || (charCount == 10) )` avbryter man inmatningen då användaren tryckt på return-tangenten eller matat in mer än 10 tecken. Därefter lagras ett strängslutstecken `s[charCount] = '\0';`.

```
void string_in( void ) /* get input to global string s[] */
{
    char ChCount, c;
    for( ChCount = 0; ; ChCount++ )
    {
        c = getchar( ); /* input 1 character */
        s[ChCount] = c; /* store the character */
        putchar( c ); /* echo the character */
        if( (ChCount == 10) || (c == '\r') ) /* end of input */
        {
            s[charCount] = '\0'; /* add "end of string" */
            return;
        }
    }
}
```



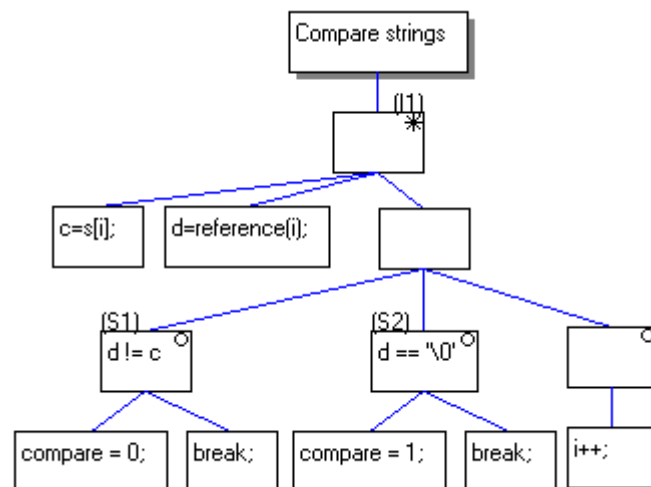
Jämför två strängar

Denna del av programmet jämför den inmatade strängen med en lagrad strängkonstant. Tecken för tecken jämförs de två strängarna, så fort man upptäcker någon olikhet så "flaggar" man detta med att sätta `compare = 0;` och avbryta jämförelsen med `break;`.

Cc5x klarar inte komplicerade uttryck så jämförelsen måste delas upp över flera satser.

När man till slut referenssträngens 0-tecken utan att jämförelsen avbrutits, är strängarna lika och `compare = 1;`.

```
/* Compare input and reference */
for( i = 0; ; i++)
{
    c = s[i];
    d = reference(i);
    if( d != c ) { compare = 0; break; }
    /* equal until end of referens string */
    if( d == '\0' ) { compare = 1; break; }
}
```



Programmet comp62x.c

```
/* comp62x.c  compare strings, serial input/output    */

#include "16F628.h"
#pragma config |= 0x3ff0

char s[11]; /* global 10 char buffer for input string */

/* Function prototypes                                     */
void initserial( void ); /* function from seriF62x.c */
bit putchar( char ); /* function from seriF62x.c */
char getchar( void ); /* function from seriF62x.c */
void string_in( void ); /* function from stringIO.c */
char text1( char ); /* function prototype */
char text2( char ); /* function prototype */
char text3( char ); /* function prototype */
char reftxt1( char ); /* function prototype */

#include "seriF62x.c"
#include "stringIO.c"

void main( void )
{
    char i, c, d, charCount;
    bit compare;

    initserial();

    /* print out textstring text1 */
    for(i = 0; putchar( text1(i)); i++) ;

    /* get string input s[] max 10 chars */
    string_in();

    /* Compare input and reference */
    for( i = 0; ; i++)
    {
        c = s[i];
        d = reftxt1(i);
        if( d != c ) { compare = 0; break; }
        /* equal until end of referens string */
        if( d == '\0' ) { compare = 1; break; }
    }
}
```

```

if(compare == 1)
{
    /* print out textstring text2 */
    for(i = 0; putchar( text2(i)); i++) ;
}
else
{
    /* print out textstring text3 */
    for(i = 0; putchar( text3(i)); i++) ;
}
}

/*
    String constants are not implemented in CC5x.
    Use text(i) -functions instead of text[i] -arrays
*/

char text1( char x)
{
    skip(x); /* internal function CC5x. */
    #pragma return[] = "Welcome! Who is burried in Grant's tomb?" '\r'
    '\n' '\0'
}

char text2( char x)
{
    skip(x); /* internal function CC5x. */
    #pragma return[] = '\r' '\n' "Congratulations! that's correct!"
    '\r' '\n'\0'
}

char text3( char x)
{
    skip(x); /* internal function CC5x. */
    #pragma return[] = '\r' '\n' "Sorry! that's wrong." '\r' '\n' '\0'
}

char reftxt1( char x)
{
    skip(x); /* internal function CC5x. */
    #pragma return[] = "Grant" '\0'
}

```

Funktionsfilen **stringIO.c**

Funktionen för inmatning av en sträng finns i en funktionsfil.

```
/* stringIO.c string serial input and output          */
/* Include serialF84.c or seriF87x.c in main program  */
/* Declare global string s[11] in main program        */
/* Function prototypes is in "stringIO.h"             */

void string_in( void ) /* get input to global string s[] */
{
    char charCount, c;
    for( charCount = 0; ; charCount++ )
    {
        c = getchar( ); /* input 1 character          */
        s[charCount] = c; /* store the character       */
        putchar( c ); /* echo the character           */
        if( (charCount == 10) || (c == '\r' ) ) /* end of input */
        {
            s[charCount] = '\0'; /* add "end of string" */
            return;
        }
    }
}

void string_out( void ) /* serial output of global string s[] */
{
    char i;
    for( i = 0; putchar( s[i] ); i++ );
}
```

16F628

Programfiler comp62x.c,

Funktionsfilen seriF62x.c (funktionsprototyperna finns i filen seriF62x.h)

Funktionsfilen stringIO.c (funktionsprototyperna finns i filen stringIO.h)

16F84

Programfiler cmpcom84.c,

Funktionsfilen seriF84.c (funktionsprototyperna finns i filen seriF84.h)

Funktionsfilen stringIO.c (funktionsprototyperna finns i filen stringIO.h)

Lösta programexempel med PIC16F628

Seriekommunikation, använd FLASH-minnet **remem62x.c**

PIC-processorernas programminne innehåller program och konstanter. Detta kan normalt inte ändras av användaren. I många tillämpningar har man behov av att kunna modifiera programkonstanterna, för att tex anpassa programmet till avvikande utrustning, eller för att ge användaren möjlighet till personliga inställningar. Man väljer då en PIC-processor med inbyggt data-FLASH-minne. Dessa minnesceller behåller data även när spänningen slås av. Programkonstanterna används då som ursprungsvärden (default-inställningen) som sedan vid behov kan modifieras med data från FLASH-minnet.

Man behöver då rutiner för att skriva och läsa i data-FLASH-minnet. Rutinerna nedan följer steg för steg beskrivningen i Microchips manual för 16F84.

Skriva i data-FLASH-minnet

Vi antar att man lagrat en sträng i en global char array `s[]`. `'\0'`-tecknet används för att markera strängens slut.

PIC16F628 har 128 st FLASH minnesceller. Dessa adresseras med tal mellan 0 och 7F som man skriver i register `EEADR`. Data kan lämnas och hämtas i registret `EEDATA`.

Man måste undvika att ett program som spårar ur (tex på grund av en störning) kan råka skriva i FLASH-minnet. Man måste därför skriva två speciella tal efter varandra (`0x55`, `0xAA`) i register `EECON2` för varje Byte man lagrar. Det får anses osannolikt att ett urspårat program skulle kunna göra detta.

```
void puts_eedata( void )
{
    /* Put global s[] -string in EEPROM-data */
    char i, c;
    for(i = 0; ; i++)
    {
        c = s[i];
        /* Write EEPROM-data sequence */
        EEADR = i;          /* EEPROM-data adress 0x00 = 0x7F */
        EEDATA = c;         /* data to be written */
        WREN = 1;           /* write enable */
        EECON2 = 0x55;      /* first Byte in comandsequence */
        EECON2 = 0xAA;      /* second Byte in comandsequence */
        WR = 1;             /* write */
        while( EEIF == 0 ) ; /* wait for done (EEIF=1) */
        WREN = 0;           /* write disable - safety first */
        EEIF = 0;           /* Reset EEIF bit in software */
        /* End of write EEPROM-data sequence */
        if(c == '\0') break;
    }
}
```

Läsa tal från data-FLASH-minnet

Vi använder en global char array `s[]` till att lagra den sträng som vi läser från data-FLASH-minnet. Strängen i FLASH-minnet slutar med `'\0'`-tecknet, och vi avslutar strängen i `s[]` på samma sätt.

```
void gets_eedata( void )
{
    /* Get EEPROM-data and put it in global s[] */
    char i;
    for(i = 0; ;i++)
    {
        /* Start of read EEPROM-data sequence */
        EEADR = i;          /* EEPROM-data address 0x00 = 0x7F */
        RD = 1;             /* Read */
        s[i] = EEDATA;       /* data to be read */
        /* End of read EEPROM-data sequence */
        if( s[i] == '\0') break;
    }
}
```

Programmet `remem62x.c` som skriver och läser data-FLASH-minnet

```
/* remem62x.c (for 16F628) stores and retrieves EEPROM-data */

#include "16F628.h"
#pragma config |= 0x3ff0

/* Function prototypes */
void initserial( void ); /* function from seriF62x.c */
bit putchar( char );    /* function from seriF62x.c */
char getchar( void );   /* function from seriF62x.c */
void puts_eedata( void ); /* function from eedatF84.c */
void gets_eedata( void ); /* function from eedtF62x.c */
void string_in( void );  /* function from stringIO.c */
void string_out( void ); /* function from stringIO.c */
char text1( char );      /* function prototype */
char text2( char );      /* function prototype */
char text3( char );      /* function prototype */

char s[11]; /* global char-array for temporary strings */
#include "eedtF62x.c"
#include "seriF62x.c"
#include "stringIO.c"
```

```

void main( void)
{
    char i;
    initserial();

    putchar('\r'); putchar('\n'); /* newline */

    /* Welcome! I recall from my memory: */
    for(i = 0; putchar( text1(i)); i++) ;

    /* Get EEPROM-data and put it in global s[] */
    gets_eedata();

    /* print out textstring s[] */
    string_out();

    putchar('\r'); putchar('\n'); /* newline */

    /* What Do you want me to remember now: */
    for(i = 0; putchar( text2(i)); i++) ;

    /* string input to global string s[] */
    string_in();

    /* Store global string s[] in EEPROM-data */
    puts_eedata();

    putchar('\r'); putchar('\n'); /* newline */

    /* Ok! I'll remember that. Now you can turn of the power. */
    for(i = 0; putchar( text3(i)); i++) ;

    putchar('\r'); putchar('\n'); /* newline */

}

/*
    String constants are not implemented in CC5x. Use
    text(i) -functions instead of text[i] -arrays
*/

char text1( char x)
{
    skip(x); /* internal function CC5x. */
    #pragma return[] = "Welcome! I recall from my memory: " '\0'
}

char text2( char x)
{
    skip(x); /* internal function CC5x. */
    #pragma return[] = "What Do you want me to remember now: " '\0'
}

char text3( char x)
{
    skip(x); /* internal function CC5x. */
    #pragma return[] = "Ok! I'll remember that."
    #pragma return[] = "Now you can turn of the power." '\0'
}

```


Funktionsfilen eedtF62x.c

```
/* eedtF62x.c read and write data flash memory for 16F628 */
/* Declare global string s[11] in main program */
/* Function prototypes in "eedtF62x.h" */

void puts_eedata( void )
{
    /* Put global s[] -string in EEPROM-data */
    char i, c;
    for(i = 0; ; i++)
    {
        c = s[i];
        /* Write EEPROM-data sequence */
        EEADR = i; /* EEPROM-data address 0x00 = 0x40 */
        EEDATA = c; /* data to be written */
        WREN = 1; /* write enable */
        EECON2 = 0x55; /* first Byte in comandsequence */
        EECON2 = 0xAA; /* second Byte in comandsequence */
        WR = 1; /* write */
        while( EEIF == 0 ) ; /* wait for done (EEIF=1) */
        WREN = 0; /* write disable - safety first */
        EEIF = 0; /* Reset EEIF bit in software */
        /* End of write EEPROM-data sequence */
        if(c == '\0') break;
    }
}

void gets_eedata( void )
{
    /* Get EEPROM-data and put it in global s[] */
    char i;
    for(i = 0; ;i++)
    {
        /* Start of read EEPROM-data sequence */
        EEADR = i; /* EEPROM-data address 0x00 = 0x40 */
        RD = 1; /* Read */
        s[i] = EEDATA; /* data to be read */
        /* End of read EEPROM-data sequence */
        if( s[i] == '\0') break;
    }
}
```

16F628

Programfil remem62x.c

Funktionsfiler eedtF62x.c, seriF62x.c, stringIO.c

16F84

Programfil rememb84.c

Funktionsfiler eedatF84.c, seriF84.c, stringIO.c

Lösta programexempel med PIC16F628

Utskrift av tal

Alla processorer arbetar internt med binära tal. För att presentera resultatet av programmets beräkningar för användaren måste man kunna skriva ut tal med vårt vanliga decimala positionssystem. Det är också nödvändigt att kunna skriva ut vad som finns i olika register när man håller på med att avlusa sin programkod. Här nedan följer ett antal funktioner för omvandling mellan strängar och tal samt för utskrift av strängar.

Funktionerna kan se korta ut, men eftersom de bygger på divisioner som ju kräver *många* maskinkodsinstruktioner att utföra, så är det en god ide att bara ta med de funktioner som för tillfället behövs.

Utskrift av ett 8-bitarstal utan tecken, funktionen `chartoa()`

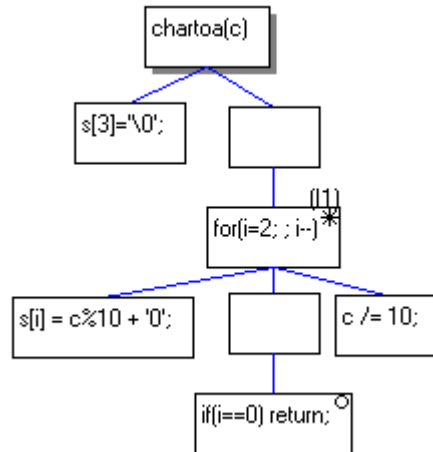
Antag att ett Byte-register (ett 8-bitarsregister) innehåller talet 123 (det vill säga binärtalet 01111011). För att skriva ut talet på en terminal eller bildskärm behöver vi "stycka upp" det i ental, tiotal och hundratal och uttrycka dessa med ASCII-koden, det vill säga som tecknen '1' '2' och '3'.

Först delar vi 123/10. Då får vi 12 och resten 3. Resten är således den sökta entalssiffran. Delar vi sedan 12/10 får vi 1 och resten 2. Resten 2 är nu tiotalssiffran. Delar man 1/10 så får man 0 och resten 1, och hundratalssiffran är således 1. Med C skriver man:

```
/* chartoa: convert "char" unsigned 8 bit variable c
   to ASCII-characters in global string s[] */
void chartoa(char c)
{
    char i,temp;
    s[3]='\0'; /* end of string */
    for (i = 2; ;i--)
    {
        temp = c % 10; /* the remainder */
        temp += '0'; /* ASCII-code of the remainder */
        s[i]=temp; /* store */
        if (i==0) break;
        c /= 10; /* the whole number quotient */
    }
}
```

Siffrorna samlas in i en global char array `s[]`. Man börjar med att markera strängslutet med 0-tecknet `s[3] = '\0'`; . Eftersom det sedan blir många divisioner måste man se till att inkludera ett matematikbibliotek i huvudprogrammet med direktivet `#include "math16.h"`.

Heltalsdivision betecknas i C med `/` och resten får man med modulus-operatören `%`. Siffrorna har ordningsnumren *efter* varandra i ASCII-tabellen. Man kan därför skriva ASCII-koden för tex. siffran '3' som `'0'+3`. ('0' har koden 48, '3' har således koden $48+3=51$).



Hur skriver man ut resultatet?

Siffrorna finns nu lagrade i en teckensträng och man kan tex. skriva ut dem på det här sättet:

```

for(i = 0; putchar( text1(i)); i++) ; /* The number is:          */
for(i = 0; putchar( s[i]); i++) ;    /* print out textstring s[] */
putchar('\n' );
putchar('\r' );                      /* Newline                  */

```

Funktionen `text1()` inleder utskriften med en ledtext och avslutar den med ny rad. Jämför med hur man skulle skrivit detta med ANSI-C och funktionen `printf()` (som ju saknas med **Cc5x**):

```
printf("The number is: %d\n", c);
```

Omvandling 8-bitstal med tecken till sträng, funktionen `inttoa()`

Hos tal med tecken är den mest signifikanta biten teckenbit. Eftersom PIC-processorerna har bitinstruktioner kan man direkt testa ett tals tecken, för att skriva ut +/- och vid behov negera negativa tal.

```
s[0] = '+'; if (i8.7 == 1) {i8 = - i8; s[0] = '-' ;}
```

Resten av funktionen är uppbyggd som det tidigare exemplet.

```

/* inttoa: convert int variable i8 with sign */
/* to ASCII-characters in global string s[] */
void inttoa(int i8)
{
    char i,temp;
    s[4] = '\0';
    s[0] = '+'; if (i8.7 == 1) {i8 = - i8; s[0] = '-' ;}
    for (i = 3; ;i--)
    {
        temp = i8 % 10;
        temp += '0';
        s[i]=temp;
        if (i==1) break;
        i8 /= 10;
    }
}

```

Funktionerna `unslongtoa()` och `longtoa()`

16-bitarsvariabler kan omvandlas till strängar med funktionerna:

```
void unslongtoa(unsigned long);
void longtoa(long);
```

Testprogram för utskrift av tal `num2a62x.c`

```
/* num2a62x.c converts variables to ASCII-strings */

#include "16F628.h"
#pragma config |= 0x3ff0
#include "math16.h"

/* function prototypes */
void initserial( void );           /* function from seriF62x.c */
bit putchar( char );              /* function from seriF62x.c */

void chartoa( char );             /* function from ascnnum.c */
void inttoa( int );              /* function from ascnnum.c */
void unslongtoa( unsigned long ); /* function from ascnnum.c */
void longtoa( long );            /* function from ascnnum.c */
void printout( void );           /* function prototype */
char text1( char );              /* function prototype */

char s[11]; /* 10 char global buffer */
#include "seriF62x.c"
#include "ascnum.c"
```

```

void main( void)
{
    char i,c;
    int i8;
    long i16;
    unsigned long u16;

    initserial();

    /* print out char variable */
    c = 123; /* load number in variable */
    chartoa(c); /* convert number to textstring s[] */
    printout(); /* printout text1, number in s[], and newline */

    /* print out int variable */
    i8 = -123; /* load number in variable */
    inttoa(i8); /* convert number to textstring s[] */
    printout(); /* printout text1, number in s[], and newline */

    /* print out unsigned long variable */
    u16 = 12345; /* load number in variable */
    unslongtoa(u16); /* convert number to textstring s[] */
    printout(); /* printout text1, number in s[], and newline */

    /* print out long int variable */
    i16 = -12345; /* load number in variable */
    longtoa(i16); /* convert number to textstring s[] */
    printout(); /* printout text1, number in s[], and newline */
}

/*
  Functions
*/

/* Formatted print, leading string, number, and newline */
void printout( void )
{
    char i;
    for(i = 0; putchar( text1(i)); i++) ; /* The number is: */
    for( i = 0; putchar( s[i] ); i++); /* print out string s[] */
    putchar('\r'); putchar('\n'); /* Newline */
}

/*
  String constants are not implemented in CC5x.
  Use text(i) -functions instead of text[i] -arrays
*/

char text1( char x)
{
    skip(x); /* internal function in CC5x. */
    #pragma return[] = "The number is: " '\0'
}

```

Funktionsfilen ascnum.c

```
/* ascnum.c convert between variables and ASCII-string */
/* Include processor header file in main program */
/* Include "math16.h" in main program */
/* Declare global string s[11] in main program */
/* Function prototypes is in ascnum.h */

/* FROM BINARY NUMBERS TO ASCII-STRINGS */
/* chartoa: convert "char" unsigned 8 bit variable c
   to ASCII-characters in global string s[] */
void chartoa(char c)
{
    char i,temp;
    s[3]='\0';
    for (i = 2; ;i--)
    {
        temp = c % 10;
        temp += '0';
        s[i]=temp;
        if (i==0) break;
        c /= 10;
    }
}

/* inttoa: convert int variable i8 with sign
   to ASCII-characters in global string s[] */
void inttoa(int i8)
{
    char i,temp;
    s[4] = '\0';
    s[0] = '+'; if (i8.7 == 1) {i8 = - i8; s[0] = '-';}
    for (i = 3; ;i--)
    {
        temp = i8 % 10;
        temp += '0';
        s[i]=temp;
        if (i==1) break;
        i8 /= 10;
    }
}

/* unslongtoa: convert unsigned long variable u16
   to ASCII-characters in global string s[] */
void unslongtoa(unsigned long u16)
{
    char i,temp;
    s[5] = '\0';
    for (i = 4; ;i--)
    {
        temp = u16 % 10;
        temp += '0';
        s[i]=temp;
        if (i==0) break;
        u16 /= 10;
    }
}
```

```

/* longtoa: convert long int variable i16
   to ASCII-characters in global string s[] */
void longtoa(long i16)
{
    char i,temp;
    s[6] = '\0';
    s[0] = '+'; if (i16.15 == 1) {i16 = - i16; s[0] = '-';}
    for (i = 5; ;i--)
    {
        temp = i16 % 10;
        temp += '0';
        s[i]=temp;
        if (i==1) break;
        i16 /= 10;
    }
}

/* FROM ASCII-STRING TO BINARY NUMBER */
char atochar( void ) /* converts ASCII-inputstring in s[] to char */
/* Inputstring must have three digits 000 ... 255 */
{
    char number, digit;
    digit = s[2]; /* ones digit */
    digit = digit - '0';
    number = digit;

    digit = s[1]; /* tenths digit */
    digit = digit - '0';
    digit = digit*10;
    number = number + digit;

    if( s[0]=='1')
    {
        /* hundreds digit */
        number = number + 100;
    }
    if( s[0]=='2' )
    {
        /* hundreds digit */
        number = number + 200;
    }

    return number;
}

```

16F628

Programfil num2a62x.c

Funktionsfiler seriF62x.c, stringIO.c

16F84

Programfil numtoa84.c

Funktionsfiler seriF84.c, stringIO.c

Lösta programexempel med PIC16F628

Inmatning av en char, 8 bitar utan tecken

Alla processorer arbetar internt med binära tal. För att användaren ska kunna mata in tal till processorn behövs rutiner för omvandling mellan text-strängar med decimala tal till binära tal.

Omvandlingen från ASCII-tecken till en char har en del likheter med omvandling i den motsatta riktningen. Det kan därför vara lämpligt att först studera exemplet *Utskrift av tal*.

Rutinen nedan gäller inmatning av en char (ett 8-bitarstal utan tecken, max 255). För att förenkla rutinen så matar man alltid in tre siffror, det vill säga man måste lägga till inledande 0:or vid behov. Någon inmatningsbuffer finns inte så "felskrivna" siffror kan *inte* redigeras med **delete** eller **backspace** tangenterna. Inmatningen avslutas med **return**.

Det finns inte någon felkontroll - man måste helt enkelt mata in talet på rätt sätt. Trots dessa begränsningar kan denna funktion vara ett mycket praktiskt hjälpmedel vid programutveckling.

Utgångspunkten för omvandlingsrutinen är att en global char array `S[ ]` innehåller den inmatade strängen med 0-tecknet på slutet (i stället för det **return**-tecken användaren avslutat med).

Detta har funktionen `string_in()`; i filen `stringIO.c` tagit hand om.

Omvandling från siffror till en char, `atochar()`

En char är ett mycket kort tal (0 ... 255). Entalssiffran kan direkt adderas in till char-talet. Hundratalssiffran kan bara vara 0, 1 eller 2. Om den är 1 ska 100 adderas, om den är 2 ska 200 adderas till char-talet. Det är således endast för 10-talssiffran som man behöver en multiplikation med 10.

Eftersom siffrornas koder ligger i ordningsföljd i ASCII-koden, kan man få de binära siffervärdena från ASCII-tecknen om man "drar ifrån" koden för siffran '0' (med operationen `- '0'`).


```

char atochar( void ) /* converts ASCII-inputstring in s[] to char */
{
    char number, digit;
    digit = s[2]; /* ones digit */
    digit = digit - '0';
    number = digit;

    digit = s[1]; /* tenths digit */
    digit = digit - '0';
    digit = digit*10;
    number = number + digit;

    if( s[0]=='1' )
    {
        /* hundreds digit */
        number = number + 100;
    }
    if( s[0]=='2' )
    {
        /* hundreds digit */
        number = number + 200;
    }

    return number;
}

```

Testprogram för inmatning av en char a2ch62x.c

Funktionen atochar(), enligt ovan, har placerats i filen ascnum.c där även funktionen för den motsatta omvandlingsriktningen chartoa() finns.

```

/* a2char84.c testprogram char serial input/output */

#include "16f628.h"
#include "Math16.h"
#pragma config |= 0x3ff0

char s[11]; /* Global buffer for strings */

/* function prototypes */
void initserial( void ); /* in seriF62x.c */
bit putchar( char ); /* in seriF62x.c */
char getchar( void ); /* in seri62x.c */
void delay( char ); /* in delays.c */
void string_in( void ); /* in stringIO.c */
void string_out( void ); /* in stringIO.c */
void chartoa( char ); /* in ascnum.c */
char atochar( void ); /* in ascnum.c */
char text1( char ); /* function prototype */
#include "seriF62x.c"
#include "ascnum.c"
#include "stringIO.c"
#include "delays.c"

```

```

void main(void)
{
    char ch, i;
    initserial();
    delay( 250 );
    while( 1 )
    {
        /* "Input an char (000 ... 255): " */
        for( i = 0; putchar( text1( i )); i++);

        string_in();

        ch = atochar(); /* the function we are testing ! */
        chartoa( ch ); /* test: convert back */

        putchar('\r'); putchar('\n'); /* print newline */
        string_out();
        putchar('\r'); putchar('\n'); /* print newline */
    }
}

char text1( char x )
{
    skip( x );
    #pragma return[]="Input an char (000 ... 255): " '\0'
}

```

16F628

Programfil a2ch62x.c

Funktionsfiler seriF62x.c, stringIO.c, ascnum.c, delays.c

16F84

Programfil a2char84.c

Funktionsfiler seriF84.c, stringIO.c, ascnum.c, delays.c

Includefiler och testprogram till laborationerna

Funktionsfiler		
Funktionstyp:	För processortyp:	
	16F84	16F62x (16F628)
Tidsfördröjningsfunktioner delay() delay10()	delays.c (delays.h) använder TIMER0	
Seriekommunikation 9600 Baud initserial() getchar() putchar()	seriF84.c (seriF84.h) använder mjukvara serie ut RB0 serie in RB1	seriF62x.c (seriF62x.h) använder inbyggd USART serie ut TX/RB2 serie in RX/RB1
Omvandling mellan ASCII-sträng och variabler chartoa() inttoa() unslongtoa() longtoa() atochar()	ascnum.c (ascnum.h) använder global char array S[]	
Inläsning och utmatning av ASCII- sträng string_in() string_out()	stringIO.c (stringIO.h) använder global char array S[]	
Lagra och hämta sträng som EEPROM-data puts_eedata() gets_eedata()	eedatF84.c (eedatF84.h) använder global char array S[]	eedtF62x.c (eedtF62x.h) använder global char array S[]

Enkla testprogram		
Testprogram:	För processortyp:	
	16F84	16F62x (16F628)
“Hello World!” Hur skriver man ut strängkonstanter?	hello84.c	hello62x.c seriF62x.c delays.c
Fungerar seriekommunikationen? Syns det man skriver på skärmen? getchar() putchar()	echo84.c	Echo62x.c seriF62x.c
Lagra och hämta sträng som EEPROM-data	rememb84.c eedatF84.c stringIO.c seriF84.c	Remem62x.c eedtF62x.c stringIO.c seriF62x.c delays.c
Omvandling och utskrift av olika variabler. Inmatning av tal.	numtoa84.c a2char84.c ascnum.c	num2a62x.c a2ch62x.c ascnum.c

